Index Stellaire : fiche pour Adrien

Le projet en bref

- **L'appli** : un site Django ouvert sur le téléphone. Le serveur calcule les positions des astres (Skyfield), la page affiche la carte du ciel façon Sky Guide, et l'appareil reçoit sa cible
- **Ce qu'on peut déjà faire** : regarder le ciel en temps réel ou accéléré, tourner et zoomer, chercher un astre, ouvrir sa fiche, demander au bras de le pointer, suivre l'état de l'appareil, voir la figure de la constellation regardée.

Partie	Contenu	État
0 à 2	Projet, base des astres, calculs Skyfield et API du ciel	fait
3.1 à 3.4	Carte, horloge virtuelle, fiche d'un astre	fait
4.1 et 4.2	Appareil : API, simulateur, bouton « Pointer », réticule	fait
3.5	Look Sky Guide : vue, zoom, horizon, panneaux, constellations, illustrations	fait
5	Code de session pour protéger le pilotage du bras	prochaine étape
6	Suivi continu, ISS, trajectoires longues	à faire
7	Événements visibles à l'œil nu	à faire
8 et 9	Recherche et planification, journal d'observation	à faire
10	Contenu vérifié : descriptions et images libres de droits	à faire
11 et 12	Wi-Fi de l'ESP32, appli installable et notifications	à faire
13	Connexion au vrai matériel, GPS	à faire
14	Durcissement pour la production	à faire

Lancer l'appli

Le plus simple pour commencer : je lance le serveur sur mon PC et tu ouvres l'appli sur ton téléphone, connecté au même Wi-Fi. Pour la lancer chez toi, voici les étapes sur Windows.

Sur ton PC

1. Installer Python 3.12 et Git.
2. Récupérer le projet : `git clone <lien du dépôt>`.
3. Créer l'environnement : `python -m venv .venv`, puis `.venv\Scripts\activate`.
4. Installer les dépendances : `pip install -r requirements.txt`
5. Préparer la base : `python manage.py migrate`, puis `python manage.py createsuperuser` pour l'administration. Je te passe les astres déjà saisis.
6. Lancer : `python manage.py runserver`, puis ouvrir http://127.0.0.1:8000 dans le navigateur.
7. Simuler le bras : dans un second terminal, `python manage.py simuler_appareil --url http://127.0.0.1:8000 --jeton <jeton>`. Le jeton de l'appareil se lit dans l'administration (/admin).
8. Avant de pousser du code : `python manage.py test` doit afficher 32 tests OK.

Sur ton téléphone

1. Le PC qui fait tourner le serveur et le téléphone sont sur le même Wi-Fi.
2. Sur le PC : `ipconfig` pour lire l'adresse IPv4 (par exemple 192.168.1.42), l'ajouter à `ALLOWED_HOSTS` dans `index_stellaire/settings.py`, puis `python manage.py runserver 0.0.0.0:8000`.
3. Sur le téléphone : ouvrir http://192.168.1.42:8000 (avec la bonne adresse).
4. Après une mise à jour du code, vider le cache du navigateur du téléphone, sinon il garde l'ancienne version.

Sécurité : tant que la Partie 5 (code de session) n'est pas faite, n'importe qui sur le Wi-Fi peut piloter le bras depuis la page.

Ce que je te propose

La priorité, c'est ton retour sur l'appli depuis ton téléphone ; ensuite, prends la mission 2

1. **Tester l'appli.** Sur ton téléphone, si possible un Android : je n'ai testé que sur iPhone. Remplis la grille de retour plus bas.
2. **Le contenu des fiches (Partie 10, sans code).** Les fiches affichent encore « Description à venir ». Pour Mars, Jupiter, la Lune, Aldébaran, Arcturus et Polaris, puis les autres astres qu'on ajoutera :
   - une description courte (5 à 8 lignes), vérifiée, avec ses sources ;
   - une image libre de droits (Wikimedia Commons, NASA, ESO), avec la source, l'auteur et la licence. Pas d'image sans licence claire.

Règles pour travailler à deux

Le contenu, le firmware et les tests sont à toi ; la carte et l'API passent par moi, pour éviter les conflits.

- **Pas de modification** de `ciel/static/ciel/carte.js` ni de l'API (`/api/...`) sans en parler avant. Le firmware et le JavaScript dépendent du format des réponses : un champ renommé casse tout.
- **Une branche Git par sujet** (par exemple `contenu-planetes` , `firmware-sync`), puis on fusionne ensemble.
- **Les tests passent avant chaque envoi** : `python manage.py test` .
- **Contenu** : seulement des textes vérifiés et des images sous licence libre, avec la source et l'auteur notés. Jamais d'illustrations de Sky Guide par exemple : elles sont protégées.
- **La feuille de route** du projet est la référence : état d'avancement, décisions prises, pistes écartées. Je te la partage ; si une décision te semble mauvaise, dis-le, on la rediscute.

Grille de retour

Remplis la colonne Résultat directement ici ; pour chaque problème, une phrase dans Remarque et une capture envoyée à part.

Ton téléphone et ton navigateur : …

Vérification	Résultat	Remarque
La page s'ouvre : horizon droit au milieu, face au sud, sol en bas	▾	
Glisser tourne la vue ; l'horizon reste droit vers l'est et l'ouest	▾	
Les boutons + et − zooment ; le pincement à deux doigts aussi	▾	
Toucher une planète ouvre sa fiche (hauteur, lever, coucher, courbe)	▾	
La recherche trouve un astre et tourne la vue vers lui	▾	
Bouton de l'heure : ×600 fait tourner le ciel, « Revenir à maintenant » revient au direct	▾	

Vérification	Résultat	Remarque
Les calques coupent constellations, illustrations, Voie lactée, noms d'étoiles	▾	
La figure d'une constellation apparaît quand on la regarde, disparaît ailleurs	▾	
Toucher la pastille en haut à gauche ouvre le panneau de l'appareil	▾	
Avec le simulateur : « Pointer » fait rejoindre l'astre au réticule vert	▾	
Téléphone en paysage : pas de déformation, rien ne disparaît	▾	
Textes et boutons faciles à lire et à toucher	▾	

Impression générale : ce qui est clair, ce qui est confus, ce qui manque.

Tes idées

Ta partie : ajouter les astres et leur contenu

Tu remplis la base des astres ; l'appli les affiche ensuite toute seule (carte, recherche, fiche), sans que tu touches au code.

Où : chaque astre est un petit fichier dans le dossier `contenu/` du projet, partagé par Git. Tu le remplis soit directement dans le fichier, soit dans l'administration de l'appli (/admin, rubrique « Objets célestes ») puis tu l'exportes (voir « Travailler chacun de son côté »).

Champ	Ce qu'on y met	Exemple
Nom	le nom en français	Vénus
Catégorie	Étoile, Planète, Lune, Amas ou Satellite	Planète
Identifiant Skyfield	le nom qui sert à calculer la position (voir plus bas)	`venus`
Description	5 à 8 lignes en français, vérifiées, sources notées à part	…
Image	JPEG ou PNG libre de droits, 1 200 px de large au plus	photo NASA

Champ	Ce qu'on y met	Exemple
Constellation (pour une étoile)	la constellation à laquelle elle appartient	Orion pour Bételgeuse

Identifiants de calcul

- Planètes : `mercury`, `venus`, `mars barycenter`, `jupiter barycenter`, `saturn barycenter`, `uranus barycenter`, `neptune barycenter` ; la Lune : `moon`. Mars, Jupiter et la Lune existent déjà.
- Étoiles : `hip:` suivi du numéro Hipparcos (Bételgeuse : `hip:27989`). Attention : aujourd'hui, seules Aldébaran, Arcturus et Polaris sont calculables. Avant d'ajouter d'autres étoiles, je dois charger le catalogue Hipparcos complet. En attendant, prépare leurs textes et leurs images.
- Amas (Pléiades, Hyades) : pas encore d'identifiant de calcul, question ouverte.

Images et sources : chaque image a trois informations obligatoires, le lien de la source, l'auteur et la licence (domaine public, CC BY, CC BY-SA…). Elles s'affichent sous l'image, dans la fiche de l'astre. Pas d'image sans licence claire. Les sources de la description se notent aussi.

Travailler chacun de son côté

Chacun travaille sur son PC avec sa propre base ; ce qu'on partage par Git, ce sont les fichiers du dossier `contenu/`. Deux astres différents ne se gênent jamais, même sans être sur le même Wi-Fi.

```
contenu/
  astres/venus.json     un fichier par astre
  images/venus.jpg      son image, nommée comme l'astre
```

Exemple de fichier `contenu/astres/venus.json` :

```
{
  "nom": "Vénus",
  "categorie": "planete",
  "identifiant_skyfield": "venus",
  "description": "Vénus est la deuxième planète du Système solaire…",
  "sources": ["https://…"],
  "image": "venus.jpg",
  "image_source": "https://…",
  "image_auteur": "NASA/JPL",
  "image_licence": "Domaine public",
  "constellations": []
}
```

À chaque fois qu'on bosse dessus

1. Récupérer le travail de l'autre : `git pull`, puis `python manage.py importer_contenu` (ta base se met à jour).
2. Ajouter ou modifier un astre : dans son fichier (image dans `contenu/images/`), ou dans l'administration puis `python manage.py exporter_contenu`.
3. Vérifier dans l'appli, puis envoyer : `git add contenu`, `git commit -m "Contenu : Vénus"`, `git push`.

Situation	Ce qui se passe
Vous ajoutez chacun des astres différents	aucun conflit, Git garde tout
Vous ajoutez chacun l'image d'un astre différent	aucun conflit
Le même astre est modifié des deux côtés	Git montre le conflit dans le fichier, ligne par ligne ; on choisit ensemble
Deux images portent le même nom	conflit : toujours nommer l'image comme l'astre (`venus.jpg`)

Règles

- `db.sqlite3` et `media/` ne vont jamais dans Git (ils sont dans le `.gitignore`) : la base de chacun se reconstruit avec `importer_contenu`.
- Un astre est reconnu par son nom : ne pas renommer un astre sans prévenir.
- L'import n'efface jamais un astre ; pour en supprimer un, on en parle.